

Deep Metric Learning

The goal of this lab is to use deep metric learning to learn an image representation to perform image retrieval. The network architectures and losses from the lecture will be used. We will train a deep CNN to extract global image descriptors by using the triplet loss or the contrastive loss. Our starting point is a MobileNetV2 network that is pre-trained on ImageNet for classification. We will further train the network, i.e. fine-tune its weights to perform retrieval of bird species on the CUB-200-2011 dataset. The requirement is to implement the network architecture, the loss, and the triplet sampling with hard-negative mining.

Dataset

Training, validation and testing is performed on the CUB-200-2011 dataset comprised images of 200 bird species. In contrast to classification tasks, training, validation, and testing is performed on 3 non-overlapping sets of classes, i.e. testing for retrieval is performed on categories that are not seen during the training.

Network architecture

The network architecture consists of a fully convolutional part, the global pooling operation and vector L2 normalization. The first part maps the image to a 3D activation tensor of size $W \times H \times D$, the second part maps the 3D tensor to a vector of D dimensions, and the third part just normalizes the magnitude of the vector.

Keep the fully convolutional part of MobileNetV2, with weights that are pre-trained on ImageNet for classification, and add two options for the global pooling operation, i.e. average pooling and max pooling.

- Implement a class `GDextractor` whose constructor is called as `model = GDextractor(input_model = mobilenetv2_model, dim = 512, usemax=True)`, which implements the `forward` function so as to extract the global descriptor of image `img` by `vec = model(img)`, which is equivalent to `vec = model.forward(img)`. If `usemax` is True, the model should extract MAC descriptors with global max pooling, otherwise SPoC descriptors with global average pooling.
- The dimensionality of the representation depends on the network architecture. Add a fully connected layer right after the global pooling to reduce the dimensionality (eg from 1280 to 512).

Contrastive and triplet loss

A triplet consists of an anchor image a , an image p that is positive to the anchor, and an image n that is negative to the anchor. Their respective global descriptors are denoted by $\mathbf{x}_a$, $\mathbf{x}_p$, and $\mathbf{x}_n$, respectively. The triplet loss is formulated by

$$l(a, p, n) = \|\mathbf{x}_a - \mathbf{x}_p\|_2^2 - \|\mathbf{x}_a - \mathbf{x}_n\|_2^2 + \mu,$$

where μ is the margin hyper-parameter. The main objective is to satisfy the condition that the positive sample is closer to the anchor than the negative sample by at least the given margin. When this condition is met, the loss would become negative, but for practical reasons, we only use the positive part of the loss to train the network.

- Implement a function `triplet_loss(distances_pos, distances_neg, margin)` that gets as input the squared Euclidean distances between anchor- positive and between anchor-negative in M -dimensional vectors `distances_pos` and `distances_neg`, respectively, where M is the number of the triplets in the batch.

Triplet sampling with hard-negative mining

Mini-batch construction is performed by selecting M triplets of the form (a, p, n) , where for a given anchor a , image p is positive example that is randomly selected from the same class as a , and image n is a hard-negative. Image n is selected at random from the 30 nearest neighbors of a that are not in the same class as a . The neighbors are estimated among the whole training set by Euclidean distance of the global descriptors of the current network and are updated at the beginning of every epoch.

- Implement a function `minehard()` within the provided `CUBtriplet` class. Its result should be stored in class member `hardneg`, where `hardneg[i]` indicates the index of the hard-negative chosen for the i -th training image.

Training

Training is performed with batches of M triplets and an epoch is considered the use of all training images as anchors exactly once. The optimizer, the training augmentations and some indicative hyper-parameter values that work for our version are included in the provided code template. Some small performance improvements are observed even after the first 1-2 epochs, but for larger improvements you might need to train much longer which makes the use of GPU necessary. Each batch includes: (i) forward pass to extract global descriptors and estimate the loss, (ii) backward pass on the average (over triplets) loss, and (iii) parameter and update and setting gradients to zero for the next batch.

Validation/Testing

Validation is performed by Euclidean-distance-based retrieval and the precision at the top-ranked image is measured and reported. Testing is performed on our servers after your model

submission on a held out test set.

As a reference: SPoC descriptor before any fine-tuning achieves 52.9 precision@1 on the validation set, and 52.0 precision@1 on the held-out test set.

What you should upload ?

We are providing the overall pipeline in file `dml.py` that can be found [here](#)^[1] with some missing parts to be written by you (marked by "your code" in comments). Upload this script after adding your modifications. The CUB dataset can be download from its original [source](#)^[2]. Additionally train the network locally with triplet loss and hard-negative mining using the indicated hyper-parameter values in the template (or your preferred values if you like) and check if it is correctly improving. The performance of your training implementation will be evaluated on the test set and part of the credits are assigned if improved performance is detected.

Possible extras for further understanding (not for bonus points)

- Try different values for the margin (e.g. 0 or 0.8) and observe how it affects the training and the validation performance
- Try contrastive loss, which should be straight-forward to add in the current implementation.
- What happens if you do not use hard-negatives but pick a random negative instead?
- What happens if you do not use any random image augmentations during the training?
- Try to train for much longer (on a GPU). Can a scheduler for the learning rate help?

Hypertext Links

[1] <http://ptak.felk.cvut.cz/personal/sumapave/public/mpv/dml/dml.py>

[2] <https://data.caltech.edu/records/65de6-vp158>

[courses/mpv/labs/deep_metric_learning.txt](#) · Last modified: 2026/02/12 11:59 (external edit)

Copyright © 2026 CTU in Prague | Operated by [IT Center of Faculty of Electrical Engineering](#)
| Bug reports and suggestions [ServiceDesk CTU](#)